

STA 5635 HW 5

Palmer Swanson

February 10, 2026

Problem 1

- (1a) Loss function: $L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N f(y_i \mathbf{x}_i^T \mathbf{w}) + s \|\mathbf{w}\|_2^2$, where $f(x) = \max(0, 1 - x)$. I do not believe that the derivative of $f(x)$ exists, at least not cleanly. The portion of the loss consisting of the average of the function only contributes if the function doesn't return 0. In other words:

$$f(y_i \mathbf{x}_i^T \mathbf{w}) = \begin{cases} 0 & y_i \mathbf{x}_i^T \mathbf{w} \geq 1 \\ 1 - y_i \mathbf{x}_i^T \mathbf{w} & y_i \mathbf{x}_i^T \mathbf{w} < 1 \end{cases}$$

So for a point where $y_i \mathbf{x}_i^T \mathbf{w} < 1$, the gradient is $\frac{\partial}{\partial \mathbf{w}}(1 - y_i \mathbf{x}_i^T \mathbf{w}) = -y_i \mathbf{x}_i^T$. For a points where $y_i \mathbf{x}_i^T \mathbf{w} \geq 1$, the gradient is $\frac{\partial}{\partial \mathbf{w}}(0) = 0$. All together, the total gradient is:

$$\nabla_{\mathbf{w}} = \frac{-1}{N} \sum_{i: (y_i \mathbf{x}_i^T \mathbf{w}) < 1} (y_i \mathbf{x}_i^T) + 2s \mathbf{w}$$

We plot the training loss vs. iteration number in table 1.

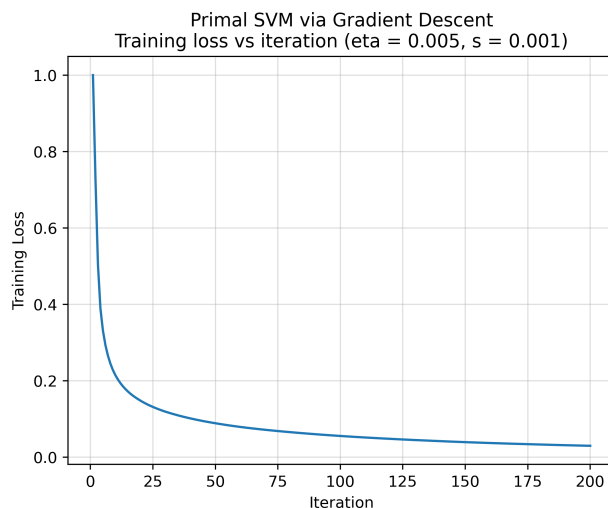


Figure 1: Primal SVM Training Loss vs. Iterations

The table outlining the training and testing misclassification errors is shown in figure 1:

Method	Train Misclass	Test Misclass
Primal SVM	0.006333	0.019000

Table 1: Train and Test Misclassification Errors, Primal SVM

Method	Train Misclass	Test Misclass	Num. Support Vectors
Linear SVM	0.000000	0.019000	1236

Table 2: Train, Test Misclassification Errors and Number of Support Vectors, Linear SVM, C=1

- (1b) Use SVM package to train linear SVM, C=1. We can report the training errors and number of support vectors in table 2: Compared to part a, the test misclassification error is the same, but the train misclassification error is slightly lower.
- (1c) Use SVM package to train a SVM with polynomial kernel of degree 2. We can report the training error and number of support vectors in table 3: Compared to part b, the

Method	Train Misclass	Test Misclass	Num. Support Vectors
Quadratic SVM	0.000167	0.025000	4744

Table 3: Train, Test Misclassification Errors and Number of Support Vectors, Quadratic SVM, C=1

number of support vectors is almost four times as great.

- (1d) Train an SVM classifier with C=1 and RBF kernels $\gamma = 10^{-i}, i \in \{1, 2, \dots, 6\}$. We plot the train and test misclassification errors vs γ in figure 2: We can also plot the number of support vectors vs γ , with a dashed line showing the number of support vectors in part b in figure 3:

- (1e) To help with part d, we consider the following table 4:

Of all the models considered, it appears as though model a and model b both obtained the lowest test error of 0.019. Of models b through d, the model in part b (quadratic SVM) has the smallest number of support vectors (1236).

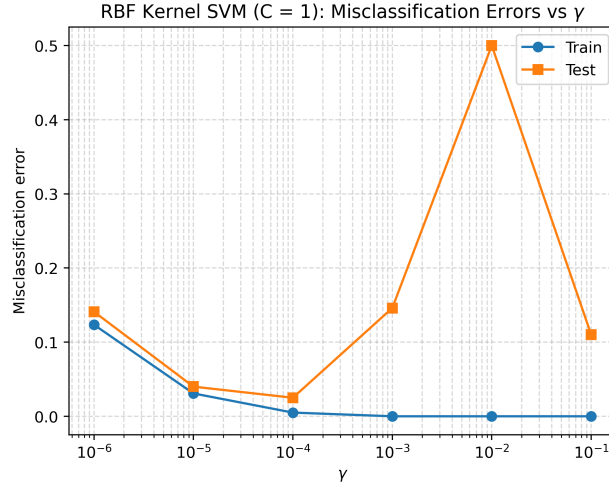


Figure 2: SVM Classifier, $C=1$, Train and Test Misclassification Errors vs γ .

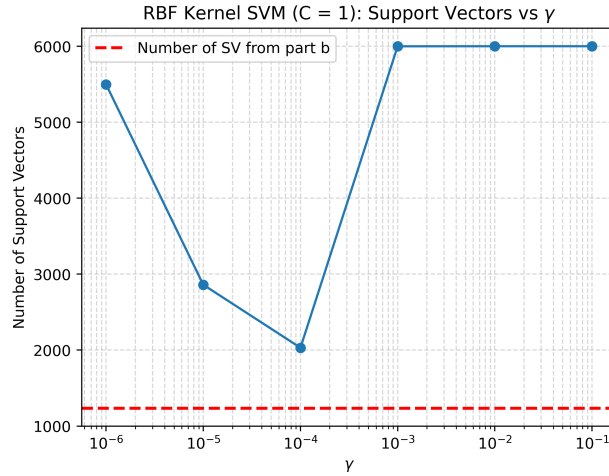


Figure 3: SVM Classifier, $C=1$, Number of Support Vectors vs γ . Dashed line indicates number of support vectors from part b (quadratic SVM).

γ	Train Error	Test Error	Support Vectors
0.100000	0.000000	0.110000	6000
0.010000	0.000000	0.500000	6000
0.001000	0.000000	0.146000	5999
0.000100	0.005000	0.025000	2031
0.000010	0.030833	0.040000	2860
0.000001	0.123500	0.141000	5497

Table 4: Train, Test Misclassification Errors and Number of Support Vectors, part d

Code

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import StandardScaler
4 import matplotlib.pyplot as plt
5 from sklearn import svm
6
7 #Q1 parameters. Gradient Descent Iterations
8 iters = 200
9 s = 0.001
10 eta = 0.005
11
12 #get train and test
13 X_train = pd.read_csv("assignments/hw05/data/gisette_train.data",
14                       delim_whitespace=True, header=None)
15
16 X_valid = pd.read_csv("assignments/hw05/data/gisette_valid.data",
17                       delim_whitespace=True, header=None)
18
19 #standardize
20 scaler = StandardScaler()
21 X_train_scaled = scaler.fit_transform(X_train)
22 X_test_scaled = scaler.transform(X_valid)
23
24 #add column of 1s to augment w
25 X_train_tilde = np.column_stack([np.ones(len(X_train_scaled)), X_train_scaled])
26 X_valid_tilde = np.column_stack([np.ones(len(X_test_scaled)), X_test_scaled])
27
28
29 #part a
30 #get N and p
31 N, p = X_train_tilde.shape
32
33 #initialize to all 0
34 w = np.zeros(p)
35
36 training_loss = []
37
38 for i in range(iters):
39     if i % 50 == 0:
40         print(i)
41         #don't need transpose?
```

```

42     #y_i*x_i*w
43     pred = y_train*(X_train_tilde @ w)
44     #the f(x) portion
45     fun = np.maximum(0.0, 1.0 - pred)
46
47     #loss function is mean of fun with the penalty term
48     #dot product of w. Don't penalize intercept
49     loss = np.mean(fun) + s*(np.dot(w[1:], w[1:]))
50
51     training_loss.append(loss)
52
53     #gradient
54     #select points in form of Boolean vector. T for being less than 1 and
55     #contributing, F for being 1 or greater
56     #and not penalizing the intercept term (w0)
57     contrib_points = pred < 1.0
58     gradient = (-1.0/N)*(X_train_tilde[contrib_points].T @
59     y_train[contrib_points])
60     gradient[1:] = gradient[1:] + (2.0*s*w[1:])
61
62     #update w
63     w = w - (eta*gradient)
64
65     #make predictions. Need a sign function
66     yhat_train = np.sign(X_train_tilde @ w)
67     yhat_test = np.sign(X_valid_tilde @ w)
68
69     #what about if we have yhat_train = 0?
70
71     train_misclass = np.mean(yhat_train != y_train)
72     test_misclass = np.mean(yhat_test != y_valid)
73
74     #plot
75     plt.figure()
76     plt.plot(np.arange(1, iters + 1), training_loss)
77     plt.xlabel("Iteration")
78     plt.ylabel("Training Loss")
79     plt.title(
80         f"Primal_SVM_via_Gradient_Descent\n"
81         f"Training loss vs iteration (eta={eta}, s={s})"
82     )
83     plt.grid(True, alpha=0.4)
84     plt.savefig("assignments/hw05/figures/primal_SVM_training_loss.png", dpi=400,
85         bbox_inches="tight")
86     plt.close()
87     #plt.show()

```

```

86 #report the errors
87 #need to save this as a table. Brackets around the elements works for some
    reason
88 #https://medium.com/practical-pandas/pandas-dictionary-to-dataframe-5-ways-to-convert-dictiona
89 error_df = pd.DataFrame({
90     "Method": ["Primal_SVM"],
91     "Train_Misclass": [train_misclass],
92     "Test_Misclass": [test_misclass]
93 })
94 print(error_df)
95
96 #need to send this table to LATEX
97 LATEX_table = error_df.to_latex(
98     index=False,
99     caption=None,
100     label=None,
101     column_format="c" * error_df.shape[1],
102     escape=False
103 )
104
105 #save latex table
106 with open("assignments/hw05/output/train_test_errors.tex", "w") as f:
107     f.write(LATEX_table)
108
109 #part b
110 #SVM
111 #https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html#sklearn.svm.Line
112 #doesn't report number of support vectors
113 #https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html#sklearn.svm.SVC
114 #does
115 svm_fit = svm.SVC(
116     C=1.0,
117     kernel="linear"
118 )
119
120 svm_fit.fit(X_train_scaled, y_train)
121
122 #make predictions
123 yhat_train = svm_fit.predict(X_train_scaled)
124 yhat_test = svm_fit.predict(X_test_scaled)
125
126 #get error
127 train_err = np.mean(yhat_train != y_train)
128 test_err = np.mean(yhat_test != y_valid)
129 num_sv = svm_fit.n_support_.sum()
130
131 #save the number of support vectors for later

```

```

132 saved_for_later = num_sv
133
134 error_df = pd.DataFrame({
135     "Method": ["Linear_SVM"],
136     "Train_Misclass": [train_err],
137     "Test_Misclass": [test_err],
138     "Num._Support_Vectors": [num_sv]
139 })
140 print(error_df)
141
142 #need to send this table to LATEX
143 LATEX_table = error_df.to_latex(
144     index=False,
145     caption=None,
146     label=None,
147     column_format="c" * error_df.shape[1],
148     escape=False
149 )
150
151 #save latex table
152 with open("assignments/hw05/output/train_test_errors_linear_svm.tex", "w") as f:
153     f.write(LATEX_table)
154
155
156 #part c, using polynomial kernel, degree 2
157 svm_poly = svm.SVC(
158     kernel="poly",
159     degree=2,
160     C=1.0
161 )
162
163 svm_poly.fit(X_train_scaled, y_train)
164
165 #make predictions
166 yhat_train = svm_poly.predict(X_train_scaled)
167 yhat_test  = svm_poly.predict(X_test_scaled)
168
169 #get error
170 train_err = np.mean(yhat_train != y_train)
171 test_err  = np.mean(yhat_test != y_valid)
172 num_sv = svm_poly.n_support_.sum()
173
174 error_df = pd.DataFrame({
175     "Method": ["Quadratic_SVM"],
176     "Train_Misclass": [train_err],
177     "Test_Misclass": [test_err],
178     "Num._Support_Vectors": [num_sv]

```

```

179 })
180 print(error_df)
181
182 #need to send this table to LATEX
183 LATEX_table = error_df.to_latex(
184     index=False,
185     caption=None,
186     label=None,
187     column_format="c" * error_df.shape[1],
188     escape=False
189 )
190
191 #save latex table
192 with open("assignments/hw05/output/train_test_errors_polynomial_svm.tex", "w")
193     as f:
194     f.write(LATEX_table)
195
196 #part d, RBF kernel estimators
197 train_errs = []
198 test_errs = []
199 num_sup_vector = []
200
201 gammas = []
202 for i in range(1,7):
203     temp = 10.0**(-i)
204     gammas.append(temp)
205
206 #looping over the gamma values
207 for gamma in gammas:
208     print(gamma)
209     svm_brf = svm.SVC(
210         kernel="rbf",
211         C=1.0,
212         gamma=gamma
213     )
214
215     svm_brf.fit(X_train_scaled, y_train)
216
217     # predictions
218     yhat_train = svm_brf.predict(X_train_scaled)
219     yhat_test  = svm_brf.predict(X_test_scaled)
220
221     # misclassification errors
222     train_err = np.mean(yhat_train != y_train)
223     test_err  = np.mean(yhat_test != y_valid)
224     num_sv = svm_brf.n_support_.sum()

```



```

225     train_errs.append(train_err)
226     test_errs.append(test_err)
227     num_sup_vector.append(num_sv)
228
229
230 #saving these results as a df
231 partd_results = pd.DataFrame({
232     r"$\gamma$": gammas,
233     "Train_Error": train_errs,
234     "Test_Error": test_errs,
235     "Support_Vectors": num_sup_vector
236 })
237
238 #need to send this table to LATEX
239 LATEX_table = partd_results.to_latex(
240     index=False,
241     caption=None,
242     label=None,
243     column_format="c" * partd_results.shape[1],
244     escape=False
245 )
246
247 #save latex table
248 with open("assignments/hw05/output/train_test_errors_num_sv_svm_partd.tex", "w")
249     as f:
250         f.write(LATEX_table)
251
252 #plot these gamma values
253 plt.figure()
254 plt.semilogx(gammas, train_errs, marker="o", label="Train")
255 plt.semilogx(gammas, test_errs, marker="s", label="Test")
256
257 #can use r to get LATEX symbols
258 plt.xlabel(r"$\gamma$")
259 plt.ylabel("Misclassification_Error")
260 plt.title("RBF_Kernel_SVM(C=1): Misclassification_Errors_vs_$\gamma$")
261 plt.legend()
262 plt.grid(True, which="both", linestyle="--", alpha=0.5)
263 plt.savefig("assignments/hw05/figures/misclass_errors_v_gamma.png", dpi=400,
264     bbox_inches="tight")
265 plt.close()
266 #plt.show()
267
268 #plot the number of support vectors vs gamma
269 plt.figure()

```

```

270 plt.semilogx(gammas, num_sup_vector, marker="o")
271 plt.axhline(
272     y=saved_for_later,
273     linestyle="--",
274     linewidth=2,
275     color="red",
276     label=f"Number_of_SV_from_part_b"
277 )
278 #can use r to get LATEX symbols
279 plt.xlabel(r"$\gamma$")
280 plt.ylabel("Number_of_Support_Vectors")
281 plt.title("RBF_Kernel_SVM(C=1):_Support_Vectors_vs_$\gamma$")
282 plt.legend()
283 plt.grid(True, which="both", linestyle="--", alpha=0.5)
284 plt.savefig("assignments/hw05/figures/num_support_vectors_v_gamma.png", dpi=400,
285             bbox_inches="tight")
286 plt.close()
#plt.show()

```